PROJECT DOCUMENTATION

Bestgen

Production-grade Arabic SaaS accounting & inventory

A bilingual ASP.NET Core MVC application engineered for the Saudi market — double-entry ledger, perpetual inventory, ZATCA-aware invoicing, role-based identity, and a dedicated Reports Center, all on a single codebase with a unified design system.

FRAMEWORK	DATABASE	LOCALES	VERSION
.NET 8 · ASP.NET Core	EF Core 8 + SQLite	Arabic · English	1.0 · May 2026

Table of contents

1 · Executive overview

Bestgen is a premium Arabic SaaS accounting + inventory web application targeted at Saudi small and mid-sized enterprises. The product replaces fragmented spreadsheets and entry-level point-of-sale tools with a single, cohesive system that handles the entire commercial cycle — from quotation through delivery, invoicing, collection, and posting to the general ledger — with full Arabic-first UX and ZATCA-aware invoicing primitives.

⚖

Double-entry GL

Every business document automatically posts a balanced journal: every debit has a matching credit, validated before save.

📦

Perpetual inventory

Sales, purchases, transfers, and counts mutate stock through an audited `StockMovement` trail.

🇸🇦

Arabic-first UX

RTL layout, native Arabic typography, bilingual labels, and locale-aware money/dates.

🔐

Role-based identity

9 built-in roles (Owner, Admin, Accountant, Sales, Purchases, Warehouse, HR, Cashier, Viewer) wired to ASP.NET Core Identity.

📊

Live reports

13 reports backed by real ledger queries — Trial Balance, P&L, Balance Sheet, aging, VAT return, customer/supplier statements, and more.

🧩

Generic CRUD framework

Master tables share a single generic `CrudController<T>` + Razor templates — adding an entity is a 5-line affair.

> **Where Bestgen fits.** A small business adopting Bestgen replaces three things at once: the manual books (or spreadsheet), a separate inventory tracker, and the discipline of producing tax-ready statements. Everything lives in one product, in one language pair, on one server.

2 · Technology stack & build

Runtime & framework

- **.NET 8** (target framework `net8.0`)
- **ASP.NET Core MVC** with Razor Views
- C# with `Nullable` and `ImplicitUsings` enabled
- Root namespace `bestgen` (lowercase) · assembly name `Bestgen`

Persistence

- **Entity Framework Core 8**
- **SQLite** file `bestgen.db` at the project root
- `EnsureCreatedAsync()` at boot — schema regenerates if the file is deleted
- `decimal(18,2)` precision configured for every monetary field

Frontend stack

- **Bootstrap 5 RTL** (`bootstrap.rtl.min.css`) for layout primitives
- **Bootstrap Icons** for iconography
- **Chart.js 4** for dashboard charts
- **Inter** + **Droid Arabic Kufi** webfonts
- Single hand-tuned stylesheet at `wwwroot/css/site.css`

Identity & security

- **ASP.NET Core Identity** with custom `ApplicationUser`
- 9 seeded roles wired through `RoleManager`
- `[Authorize]` default on every CRUD controller
- `[ValidateAntiForgeryToken]` on every POST

Build & run

```
dotnet clean
dotnet restore
dotnet build
dotnet run            # listens on http://localhost:5000 by default
```

Default seeded credentials

Email	Password	Roles	Notes
`max@bestgen.com`	`123`	Owner, Admin	Primary developer login
`admin@ledgerflow.local`	`Admin@12345`	Owner, Admin	Legacy login

The seeder calls `ResetPasswordAsync` on every run, so the credentials remain valid even after the password rules are tightened.

Database lifecycle

> **No migrations.** Bestgen relies on `EnsureCreatedAsync()`. When you change an entity, delete `bestgen.db` and re-run — the schema is recreated from the model on the next boot. Suitable for rapid iteration; needs to be replaced with EF migrations before going to production with real customer data.

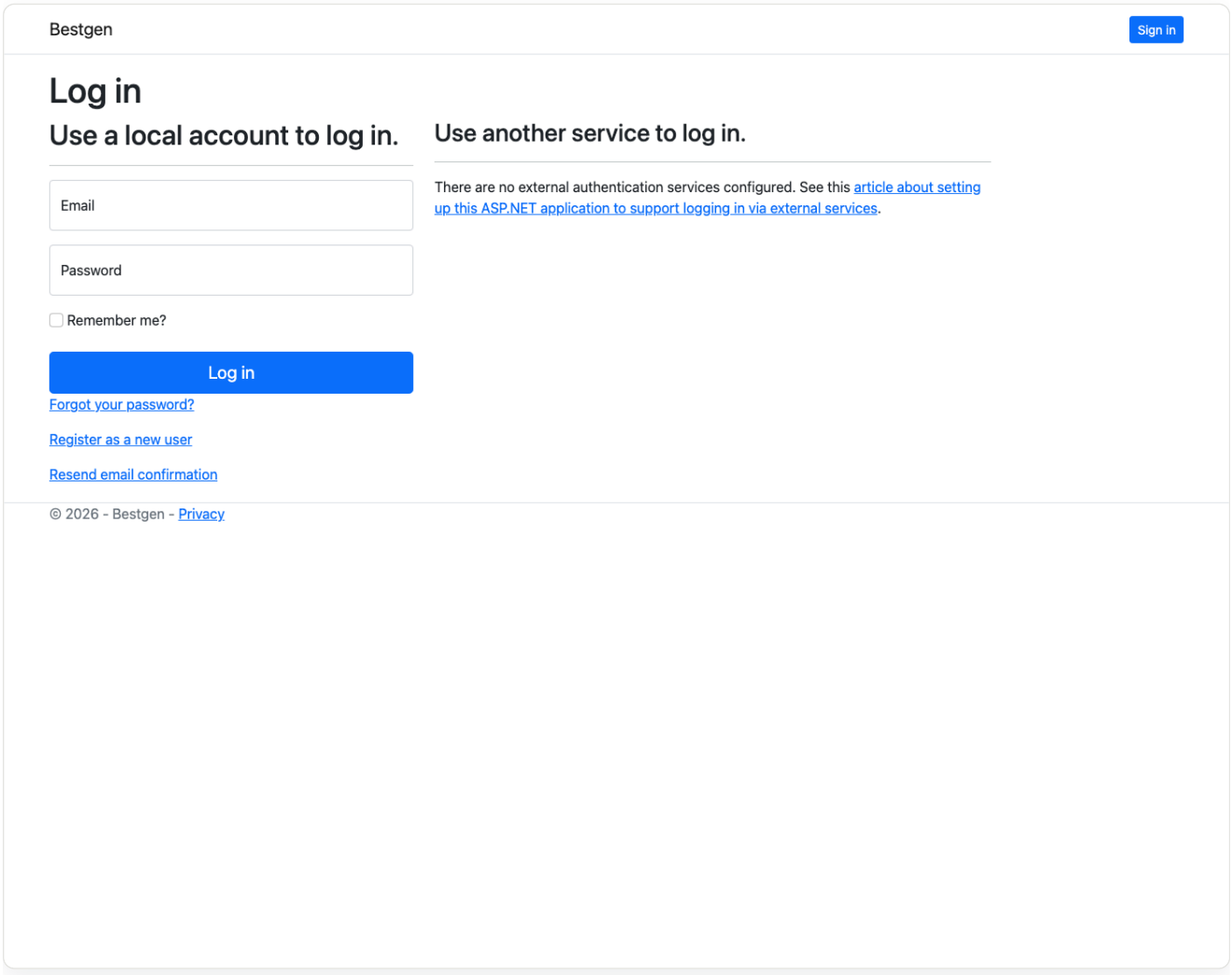
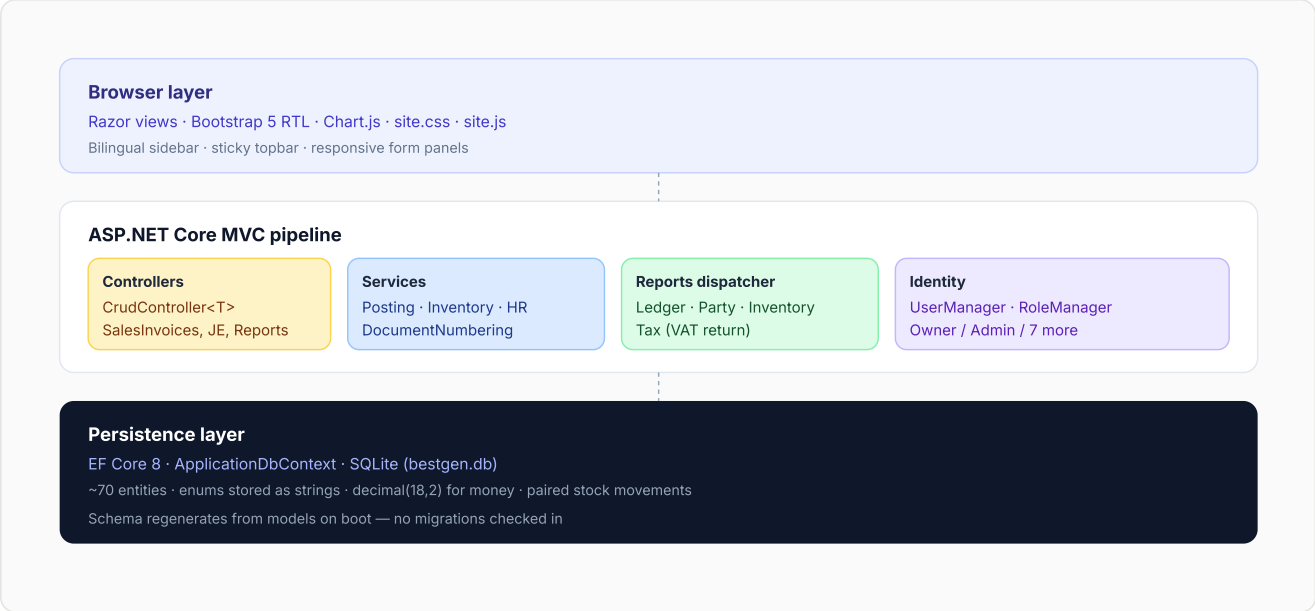


Figure 1 — Sign-in screen. Identity is configured with relaxed dev passwords; production rules are queued for Phase 9.

3 · System architecture



Layered responsibilities

- **Browser layer** renders Razor views compiled by ASP.NET. The right-side RTL sidebar, sticky topbar, and form panels are reused across every screen via shared partials.
- **Controllers** stay thin. The generic `CrudController<TEntity>` handles index/create/edit/delete for ~30 simple modules; only Sales/Purchase invoices, Journal Entries, Reports, and Settings have hand-rolled controllers.
- **Services** hold the business rules. Each transaction type has a dedicated service that orchestrates calculation → stock movement → journal posting in the right order.
- **Reports dispatcher** centralizes 13 live reports. `ReportService.RunAsync(key, filters)` returns a generic `ReportResult` structure rendered by a single Razor view.
- **Persistence** is a single EF Core `DbContext` writing to SQLite. Decimal precision, enum-to-string conversions, unique indexes, and FK delete behaviors are configured in `OnModelCreating`.

The transaction lifecycle

Every transaction module that mutates the ledger or stock follows the same three-step lifecycle, exposed through hooks on `CrudController`:

Hook	When it fires	Service contract
`BeforeCreateSaveAsync`	Before the row is added to the change tracker	`service.PrepareAsync(entity)` — generate document number, default missing fields
`AfterAddBeforeSaveAsync`	After the row is added but before `SaveChanges`	`service.PostAsync(entity)` — append journal lines, append stock movements
`AfterSaveAsync(entity, isCreate)`	After `SaveChanges` returns the new id	`service.ApplyAsync(entity)` — recalc party balance, stamp links that need the id

This pattern means every transaction is atomic: either the source document, the journal entry, the stock movement, and the party balance recalculation all commit together, or nothing does.

4 · Frontend & design system

The UI is intentionally restrained: a calm, premium palette built around a navy-and-emerald accent system, generous whitespace, and a strict typographic scale. Nothing is borrowed from competing accounting products.

Color palette (CSS variables)

Layout grammar

- **Right-side RTL sidebar** with collapsible groups and sub-icons — collapses to a 64-pixel rail on demand and re-expands on hover with a smooth animation.
- **Sticky topbar** with workspace search, language chip (AR/EN), dark-mode chip, and a user menu.
- **Page header** on every screen via `_PageHeader` partial: title + description + optional CTA.
- **Metric grid** for KPI dashboards (`.metrics-grid`), **data panel** for tables, **form panel** for create/edit, **details panel** for read-only views, **report grid** for the Reports Center.

Reusable partials

_PageHeader
Title, description, optional create button. Top of every page.

_DashboardCard
KPI cards with sparkline and delta vs. last month.

_StatusBadge
Colored badge resolved via `EntityDisplayHelper.StatusClass` .

_SearchBar
List-view filter — search + status + apply.

_EmptyState
Friendly message when a list/index has no rows.

_ActionButtons
View / Edit / Delete trio with anti-forgery on delete.

Bilingual rendering

Every view begins with `CultureInfo.CurrentUICulture` ; a tiny inline `T(en, ar)` helper picks the right string. Entity field labels, module titles, status enum translations, and report column headers all live in `Helpers/EntityDisplayHelper.cs` as bilingual dictionaries — adding a new language requires only an extra dictionary, not new views.

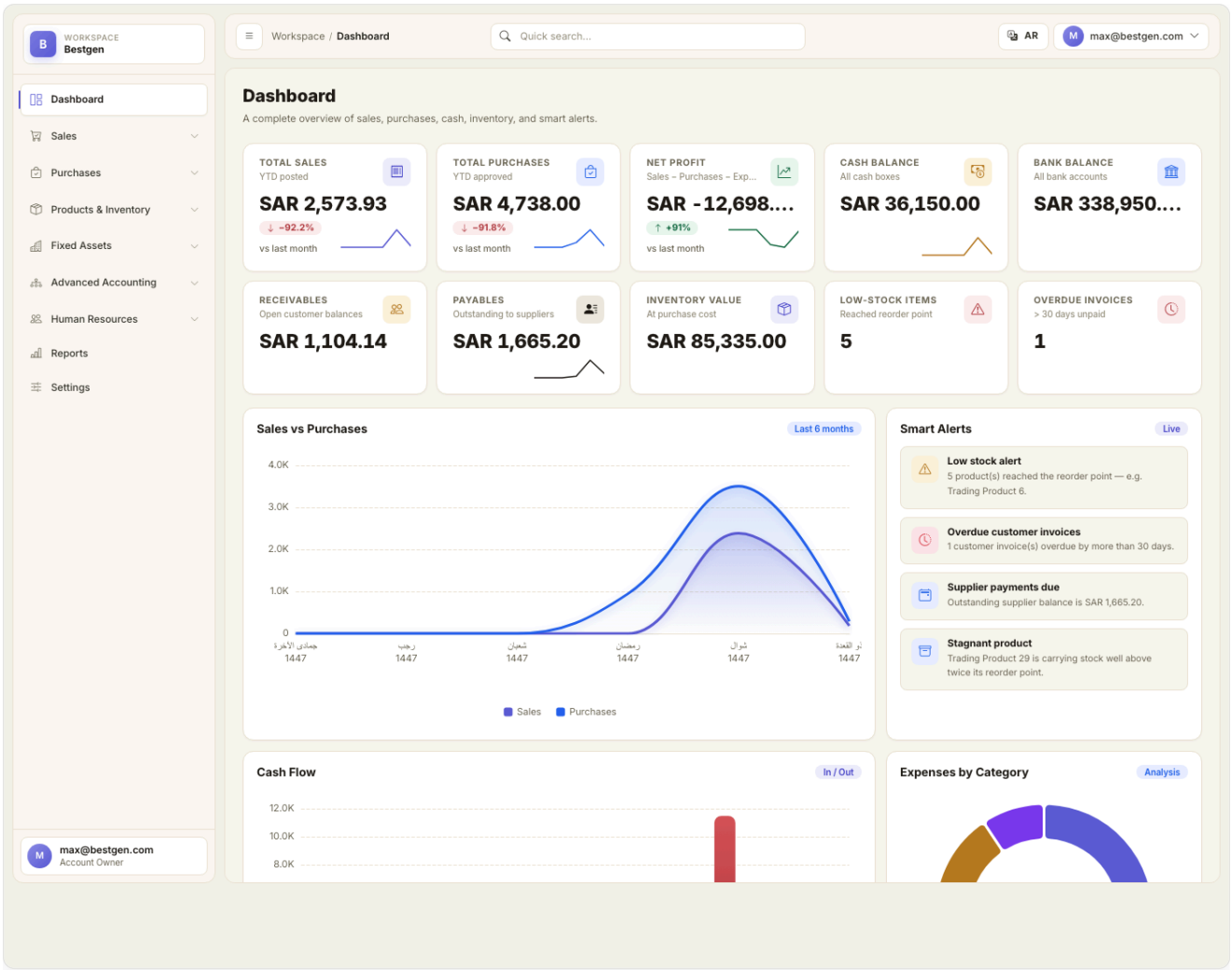


Figure 2 — Dashboard in English LTR mode. KPIs are computed by `DashboardService` from live ledger and inventory data.

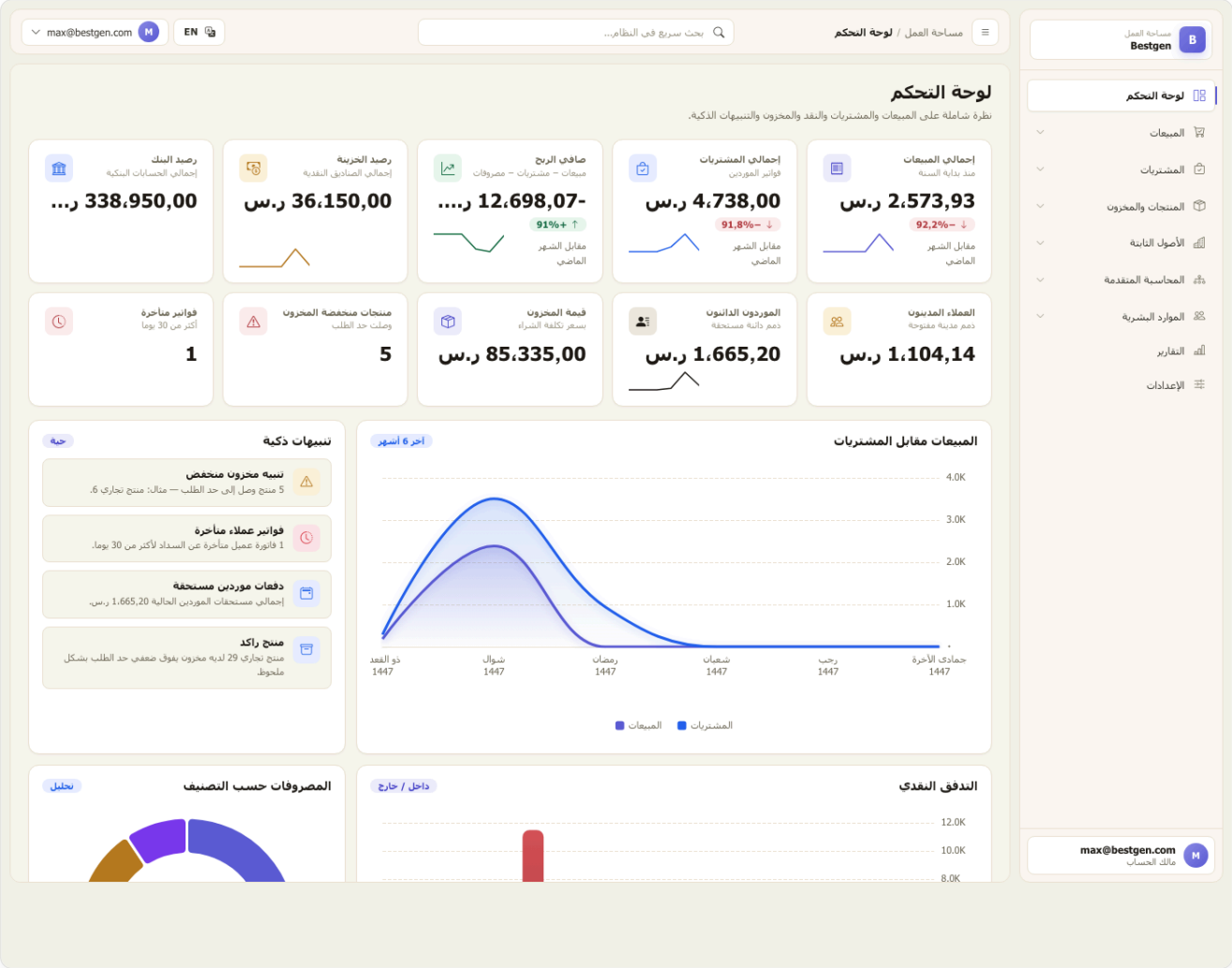


Figure 3 — The same Dashboard in Arabic RTL mode. Layout, charts, currency, and labels all flip on the language switch.

5 · Backend services & the posting engine

The backend is service-oriented. Controllers stay thin and delegate to a dedicated service per transaction type. The services share a small cluster of foundation primitives that every business operation routes through.

Foundation services

Service	Responsibility
`ChartOfAccounts`	Single source of truth for canonical account codes (Cash 1000, AR 1100, Inventory 1200, AP 2000, VAT 2100/2110, Sales 4000, COGS 5000, …). Throws `MissingAccountException` if a required code is absent.
`StockMovementService`	The only path that mutates stock. Writes paired audit rows with sign convention *positive = inbound, negative = outbound* and a typed reference (`EntityName:Id`).
`AccountingService`	Builds and validates journal entries. `BuildAndAddEntryAsync(date, sourceModule, description, lines)` validates that debits = credits and adds the entry to the change tracker.
`PartyBalanceService`	Recomputes `Customer.CurrentBalance` / `Supplier.CurrentBalance` from the primary documents — invoices minus receipts, credits, refunds, plus opening balance.
`InvoiceCalculationService`	Front-and-back recalculation of invoice subtotal, discount, VAT, and grand total — used by both Sales and Purchase invoices and validated server-side after the form posts.
`DocumentNumberingService`	Per-document numbering with format strings (`{prefix}-{yyyy}-{0000}`), annual reset, current sequence persistence.

Posting flow — Sales Invoice example

Transaction services (full inventory)

Sales side

- `SalesInvoiceService` — DR AR, CR Sales + VAT, plus COGS/Inventory pair
- `SalesReceiptService` — DR Cash/Bank, CR AR
- `SalesRefundReceiptService` — DR Sales Returns, CR Cash/Bank
- `CreditNoteService` — DR Sales Returns + VAT-Out, CR AR
- `DeliveryNoteService` — number generation only (stock stays on the invoice flow)

Purchase side

- `PurchaseInvoiceService` — DR Inventory + VAT-In, CR AP
- `SupplierPaymentService` — DR AP, CR Cash/Bank
- `PurchaseRefundReceiptService` — DR Cash/Bank, CR Purchase Returns
- `DebitNoteService` — DR AP, CR Purchase Returns + VAT-In
- `GoodsReceiptService` — number generation only

Inventory ops

- `StockTransferService` — paired stock movements, no GL impact
- `InventoryCountService` — variance posted to `5800 Inventory Variance`
- `OpeningBalanceService` — bootstraps the books against `3200 Opening Balance Equity`

HR & assets

- `PayrollService` · `EmployeeBonusService` · `EmployeeDeductionService`
- `EmployeeLoanService` — DR AR-Employees, CR Cash on disbursement
- `EmployeeReceiptService` — DR Salaries Payable, CR Cash
- `FixedAssetService` — DR Fixed Assets, CR Cash on acquisition
- `AssetRentalService` — number generation (recurring revenue deferred)

6 · Module catalog (sidebar)

The right-side sidebar groups every module into nine canonical sections. Every link resolves to a working page — empty modules render a polished empty-state, never a 404.

1 · لوحة التحكم — Dashboard	KPI grid with revenue, expenses, gross margin, AR/AP balances, stock alerts. Charts pull from `DashboardService`.	Ready
2 · المبيعات — Sales	Customers · Sales Quotations · Sales Invoices · Sales Receipts · Sales Refund Receipts · Credit Notes · Delivery Notes · Sales Pricing Policies.	Posting wired
3 · المشتريات — Purchases	Suppliers · Purchase Orders · Purchase Invoices · Supplier Payments · Purchase Refund Receipts · Debit Notes · Goods Receipts · Purchase Pricing Policies.	Posting wired

4 · المنتجات والمخزون — Products & inventory	Product Categories · Products · Warehouses · Inventory Counts · Stock Transfers.	Movements audited
5 · الأصول الثابتة — Fixed assets	Fixed Assets · Asset Tags · Asset Rentals.	Acquisition posts Depreciation Phase 9
6 · المحاسبة المتقدمة — Advanced accounting	Accounts (chart of accounts) · Journal Entries · General Receipts · Opening Balances · Reporting Dimensions.	Live
7 · الموارد البشرية — Human resources	Employees · Employee Documents · Payroll Entries · Deductions · Bonuses · Loans · Employee Receipts · Employee Requests.	All post to GL
8 · التقارير — Reports Center	13 live reports across Accounting · Sales · Purchases · Inventory · HR.	Live data
9 · الإعدادات — Settings	Company profile · Tax · Invoice numbering · Currency · Numbering policies · Branches · Users & roles · Future integrations.	All real forms

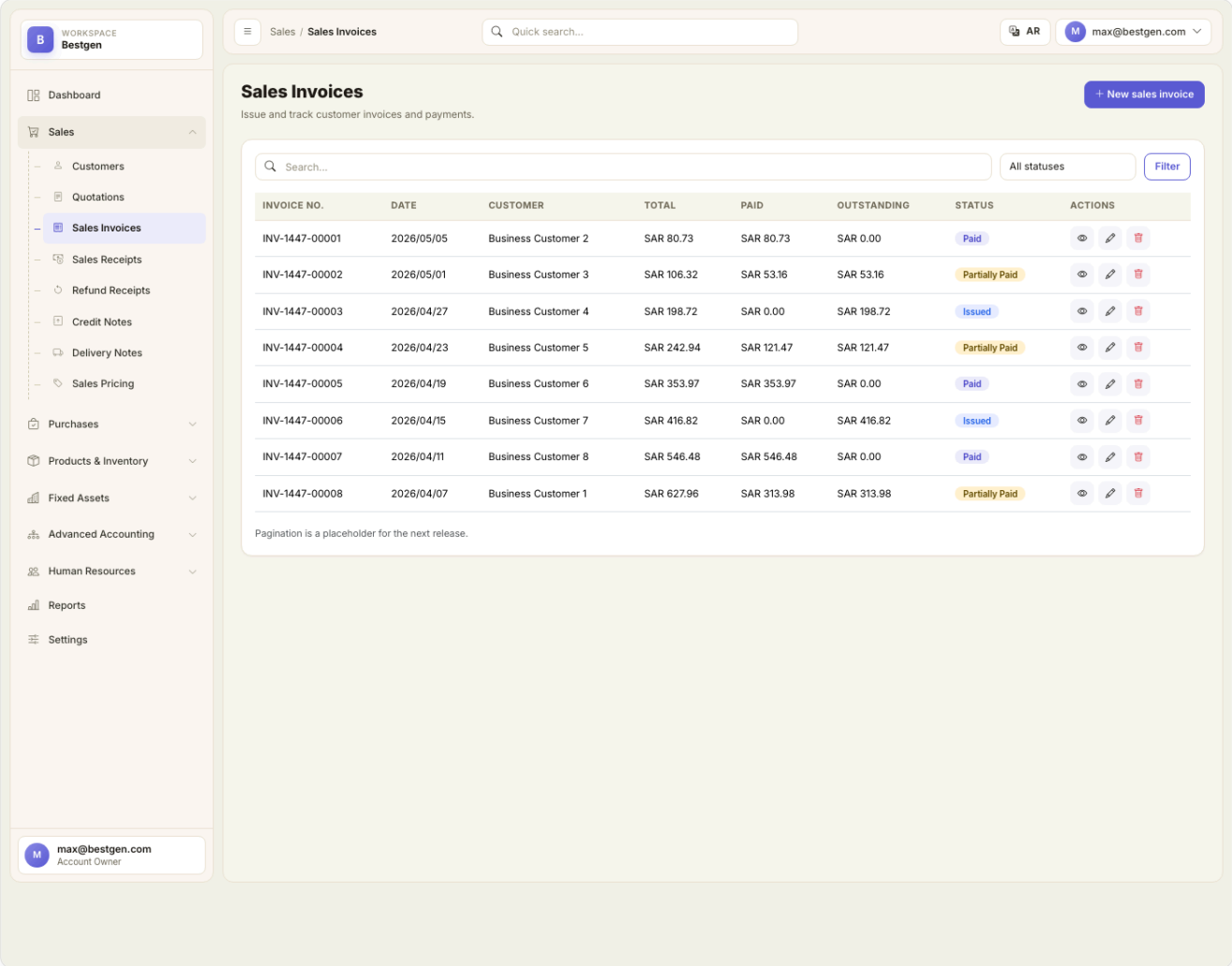


Figure 4 — Sales invoices list view, rendered via the generic data-panel template with `_SearchBar` on top and `_ActionButtons` per row.

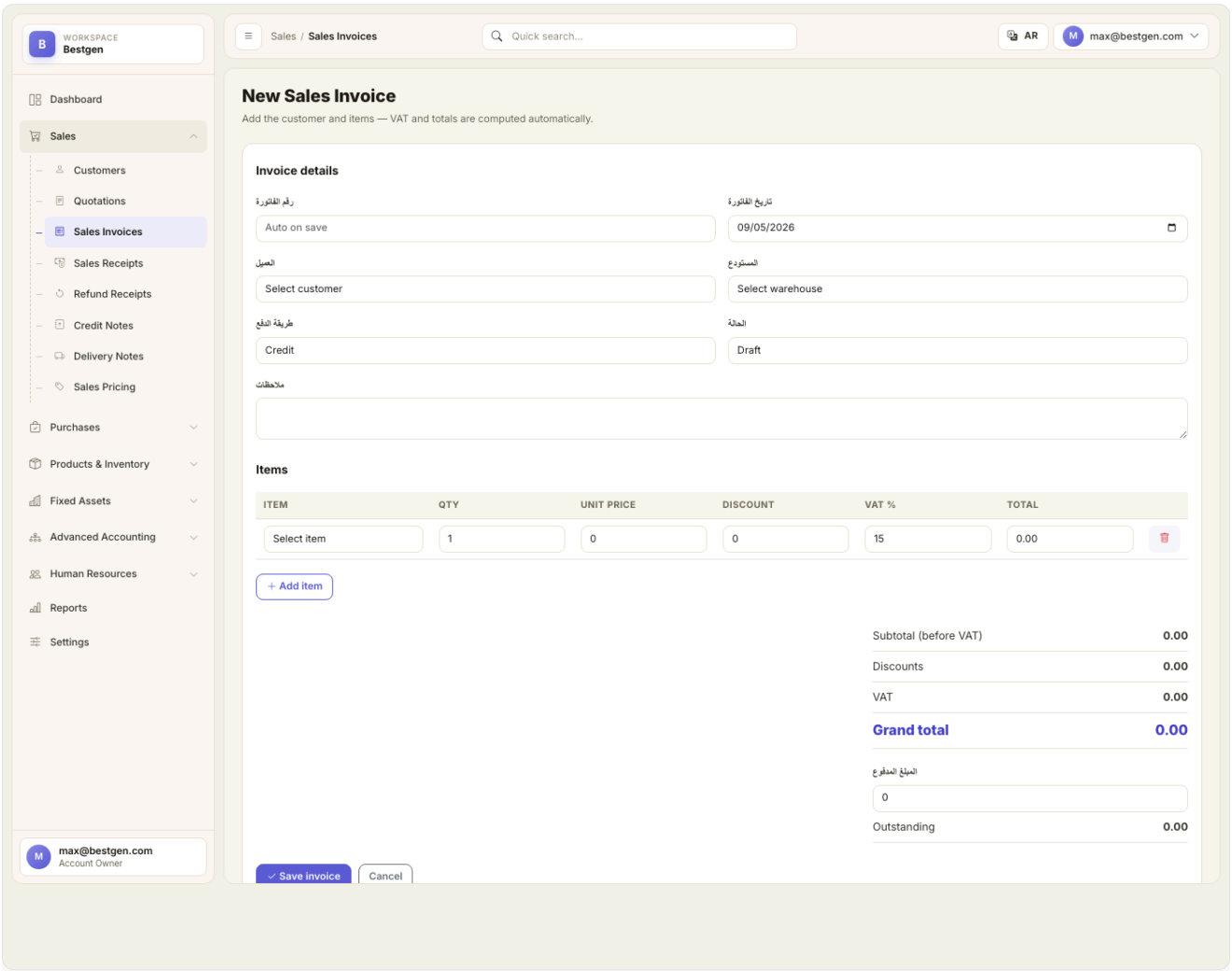


Figure 5 — Sales invoice multi-line create form. The form posts to `SalesInvoicesController.Create`, which delegates to `SalesInvoiceService` for calc + stock + journal in a single transaction.

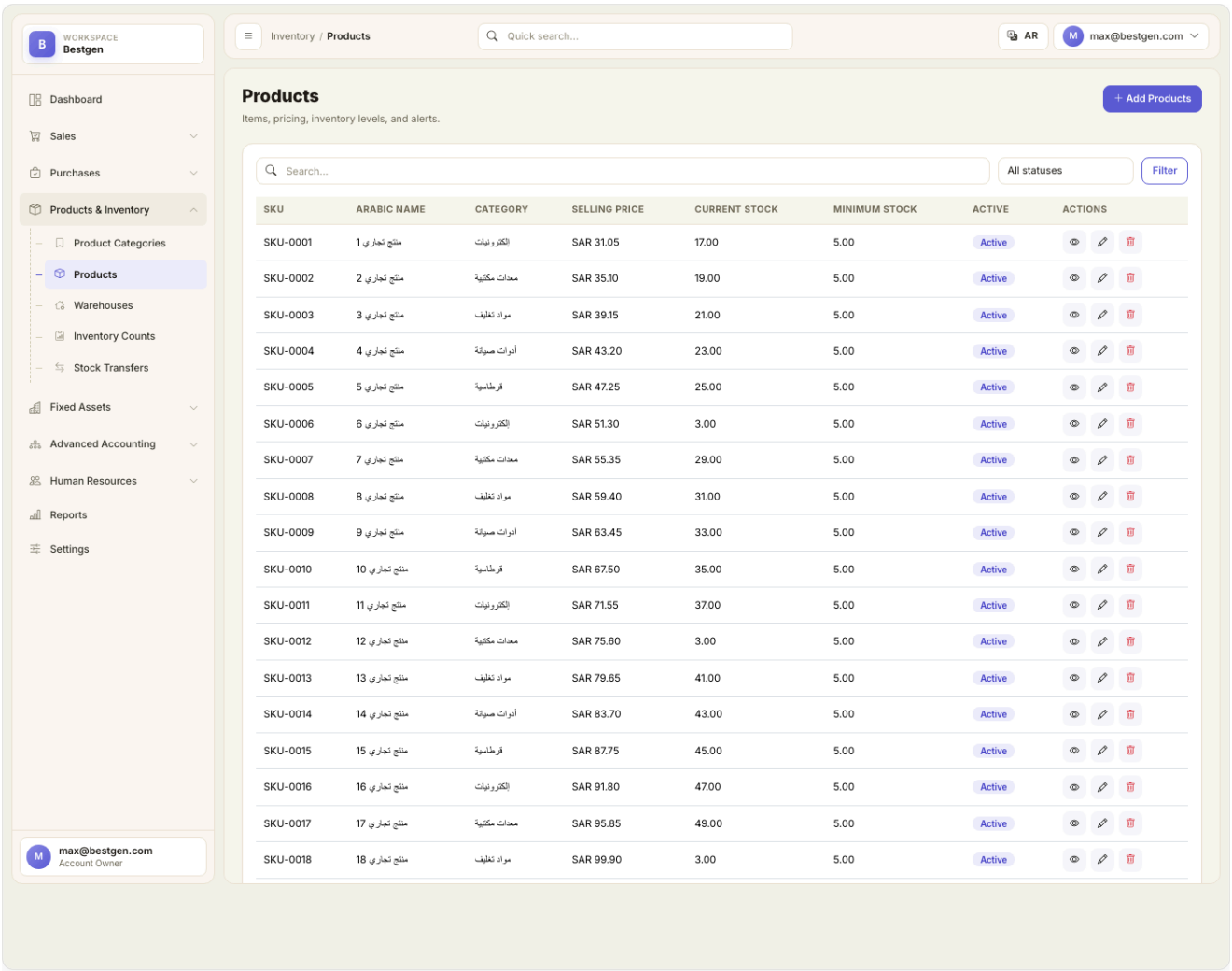


Figure 6 — Products master with current stock, sell price, and minimum levels. Adding an entity is a 5-line affair against `CrudController<T>`.

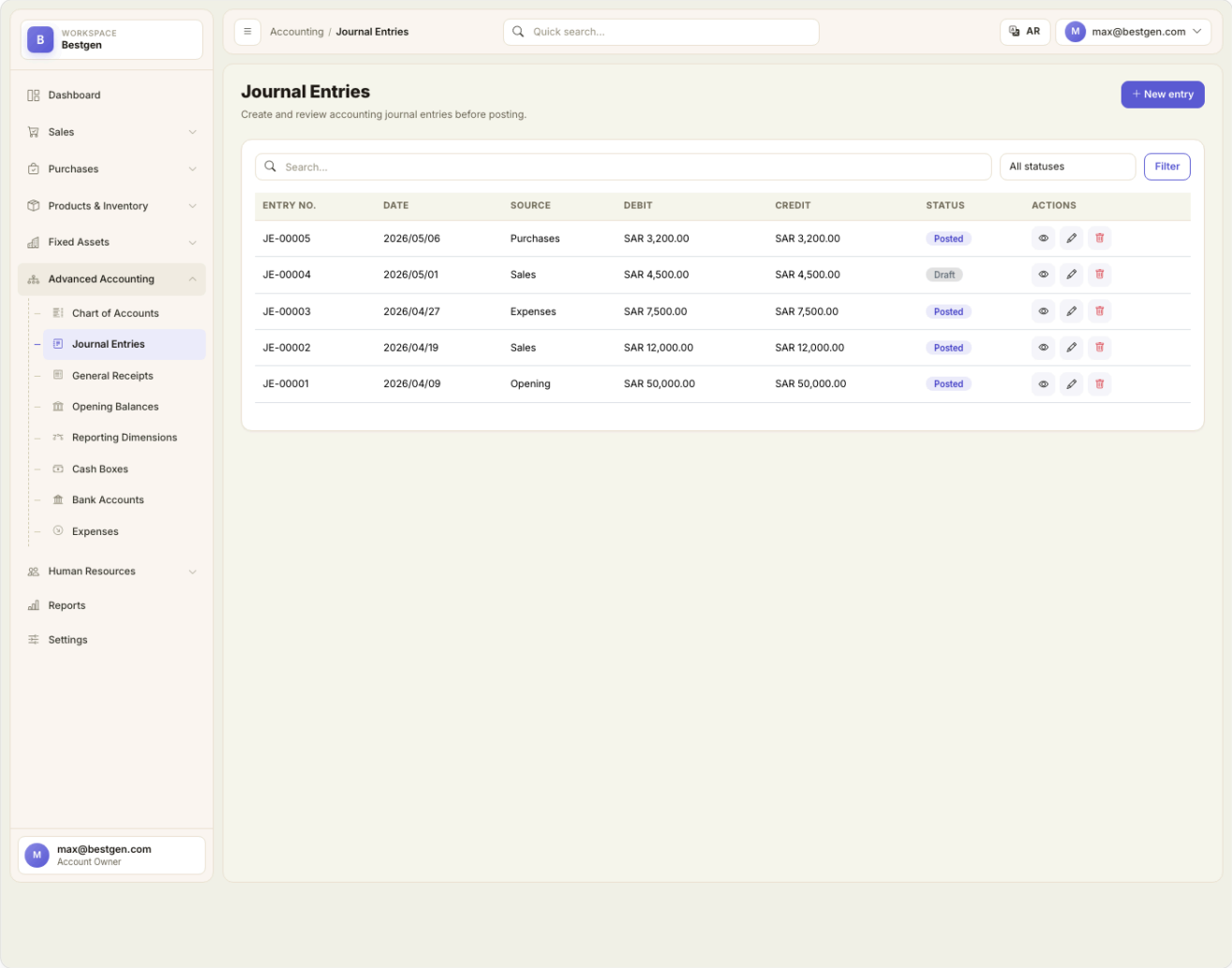


Figure 7 — Journal entries module. Manual entries route through `JournalEntryService`, which validates that debits = credits before saving.

7 · Reports Center

The Reports Center replaces what used to be 26 placeholder cards with a dispatcher pattern: `ReportService.RunAsync(string key, ReportFilters)` returns a generic `ReportResult` rendered by a single Razor view (`Views/Reports/Details.cshtml`). Adding a new report is a switch-case + a new method.

Live reports (Phase 7 — implemented)

General ledger

Account-by-account journal lines with running balance over the chosen period.

Trial balance

Sum of debits and credits per account; verification that the books are balanced.

Income statement

Revenue minus expenses for any period.

Balance sheet

Point-in-time assets vs. liabilities & equity, including current-period net income flowing into retained.

Account statement

Movement & balance for any single account.

General receipts summary

Receipt and payment vouchers grouped by category.

Customer / supplier statements

Document-level history with running balance per party.

Customer / supplier balances

Snapshot of all party balances.

Aging receivables / payables

0–30 / 31–60 / 61–90 / 91–120 / 120+ buckets.

Current stock

Per-warehouse on-hand quantity and valuation.

Stock movement ledger

Audit trail of every quantity change.

VAT return

Output VAT − input VAT, with credit/debit note adjustments.

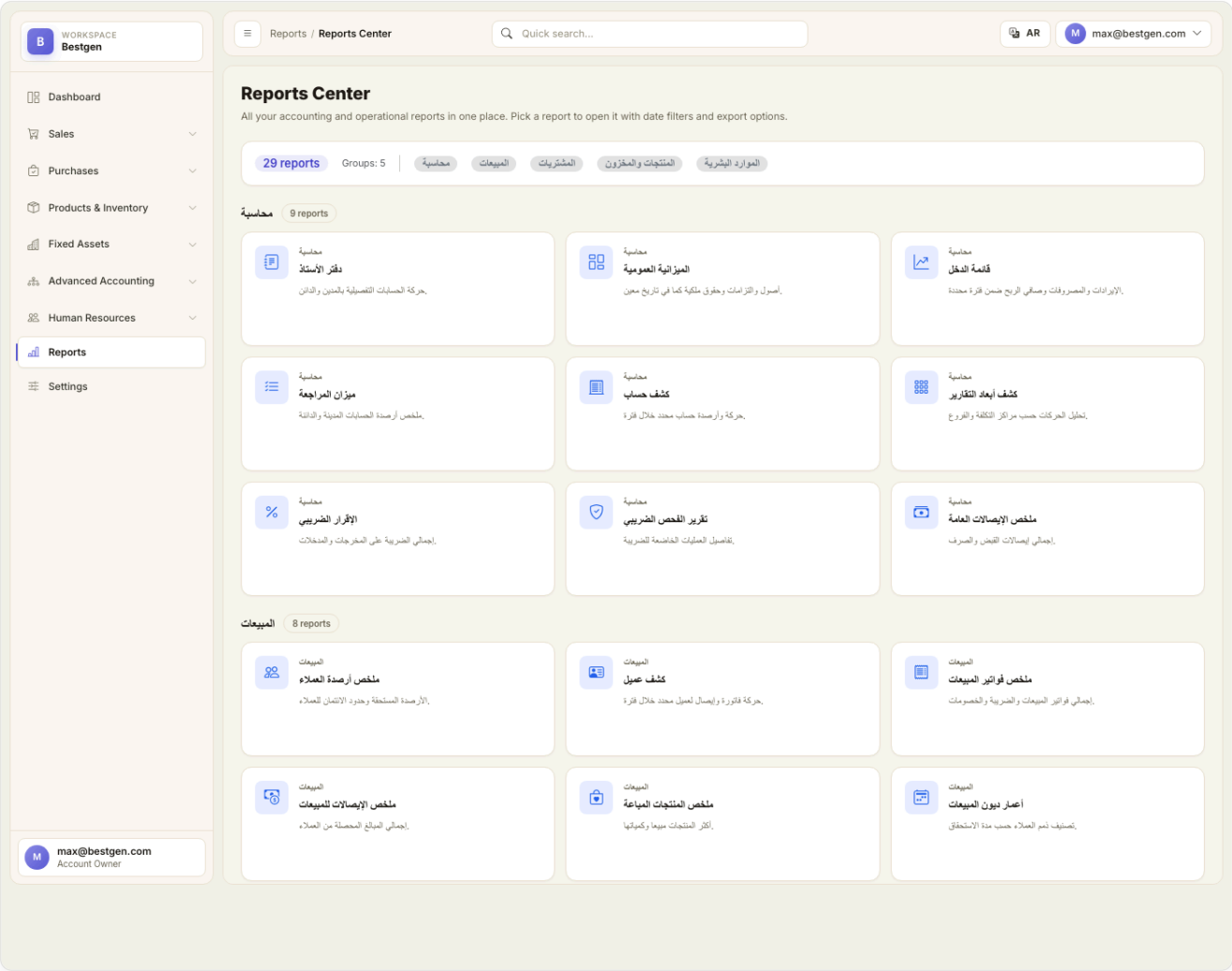


Figure 8 — Reports Center hub. Reports are grouped into Accounting · Sales · Purchases · Inventory & products · HR.

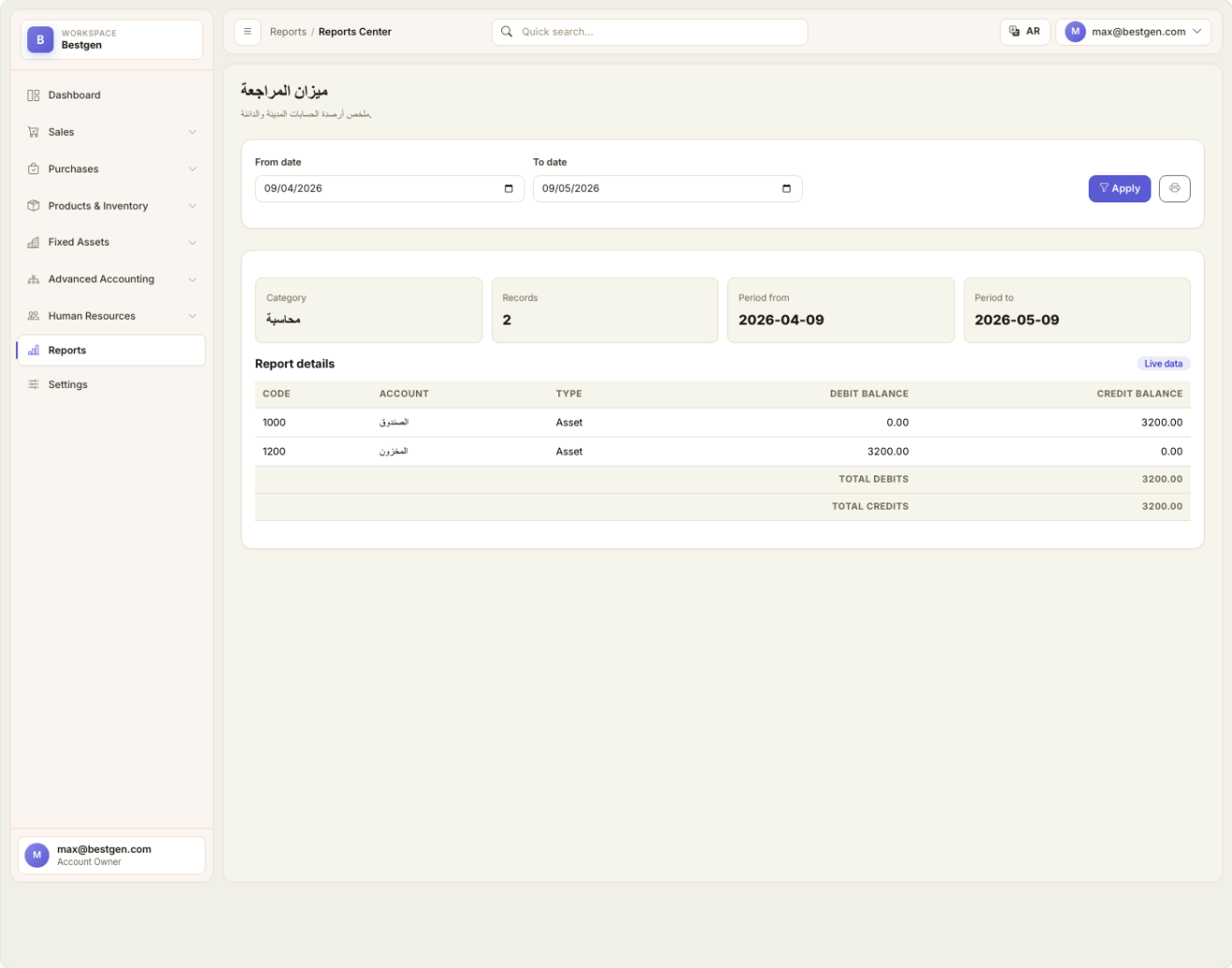


Figure 9 — Trial Balance report. Date filters route through `ReportFilters`; the column/row/total shape is universal across reports.

8 · Settings & identity

Settings is split into eight dedicated screens, each backed by a real form (no placeholders left).

Page	What it controls
Company profile	Names (AR/EN), VAT number, commercial registration, address, country, default VAT rate, invoice prefixes, logo upload.
Tax settings	Default VAT rate, VAT number, ZATCA enable flag, ZATCA taxpayer ID, invoice QR toggle.
Invoice settings	Sales/purchase invoice prefixes, AR/EN footer text, QR display toggle.
Currency settings	Base ISO currency code (default `SAR`) and display symbol.
Numbering settings	List of `NumberingPolicy` rows — one per document type, with prefix, format pattern, current sequence, annual-reset flag.
Branches	Branch CRUD (code, AR/EN name, city, address, phone, manager, active flag). Wired into the generic CRUD framework.
Users & permissions	Identity-backed CRUD: list users with role badges and lock state, create user with role assignment, edit user (rename / change roles / reset password), delete user.
Future integrations	Roadmap card grid: payment gateways, bank reconciliation, WhatsApp Business, ZATCA Phase 2, delivery providers, transactional email.

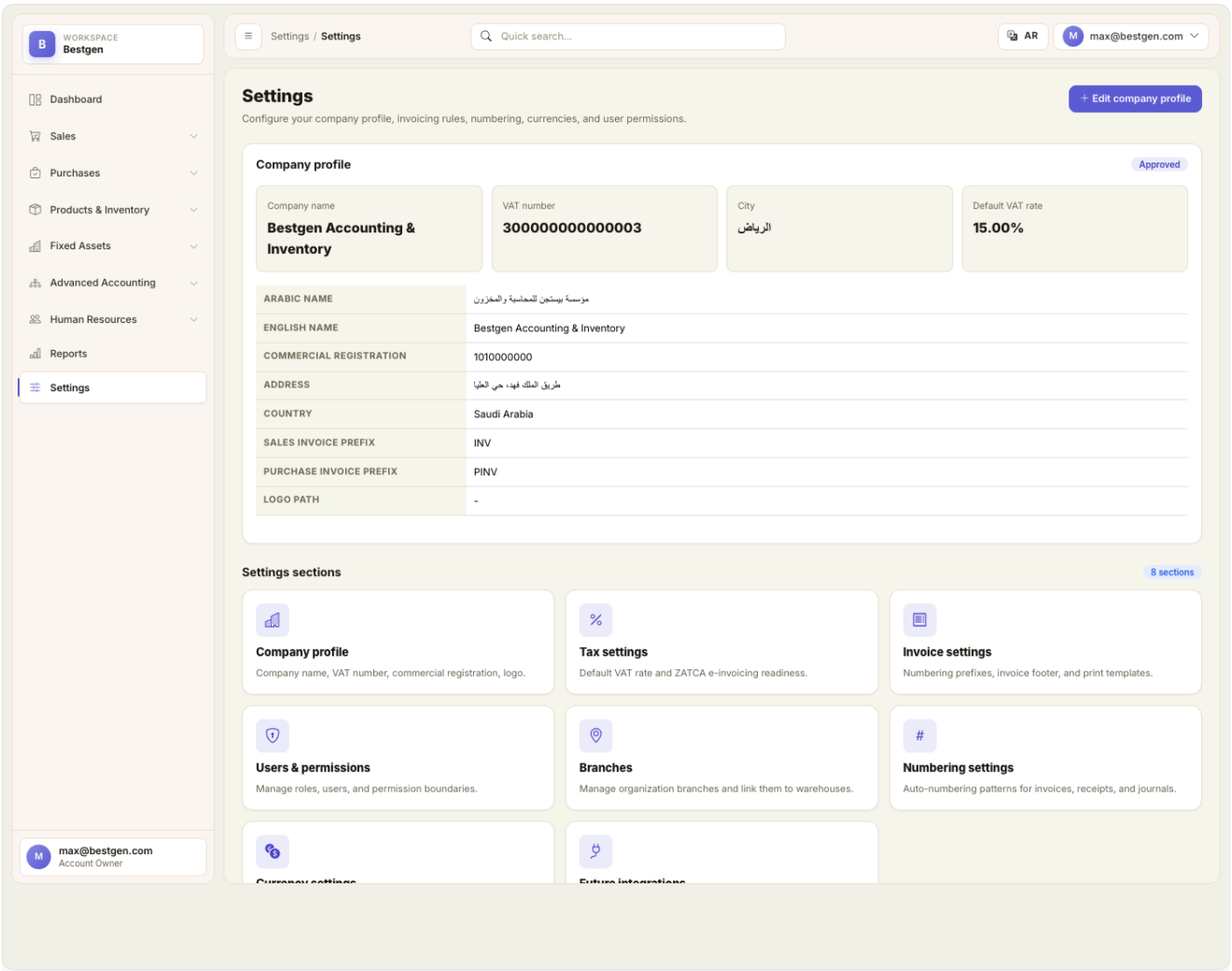


Figure 10 — Settings hub. Each card opens a real form; nothing is a placeholder.

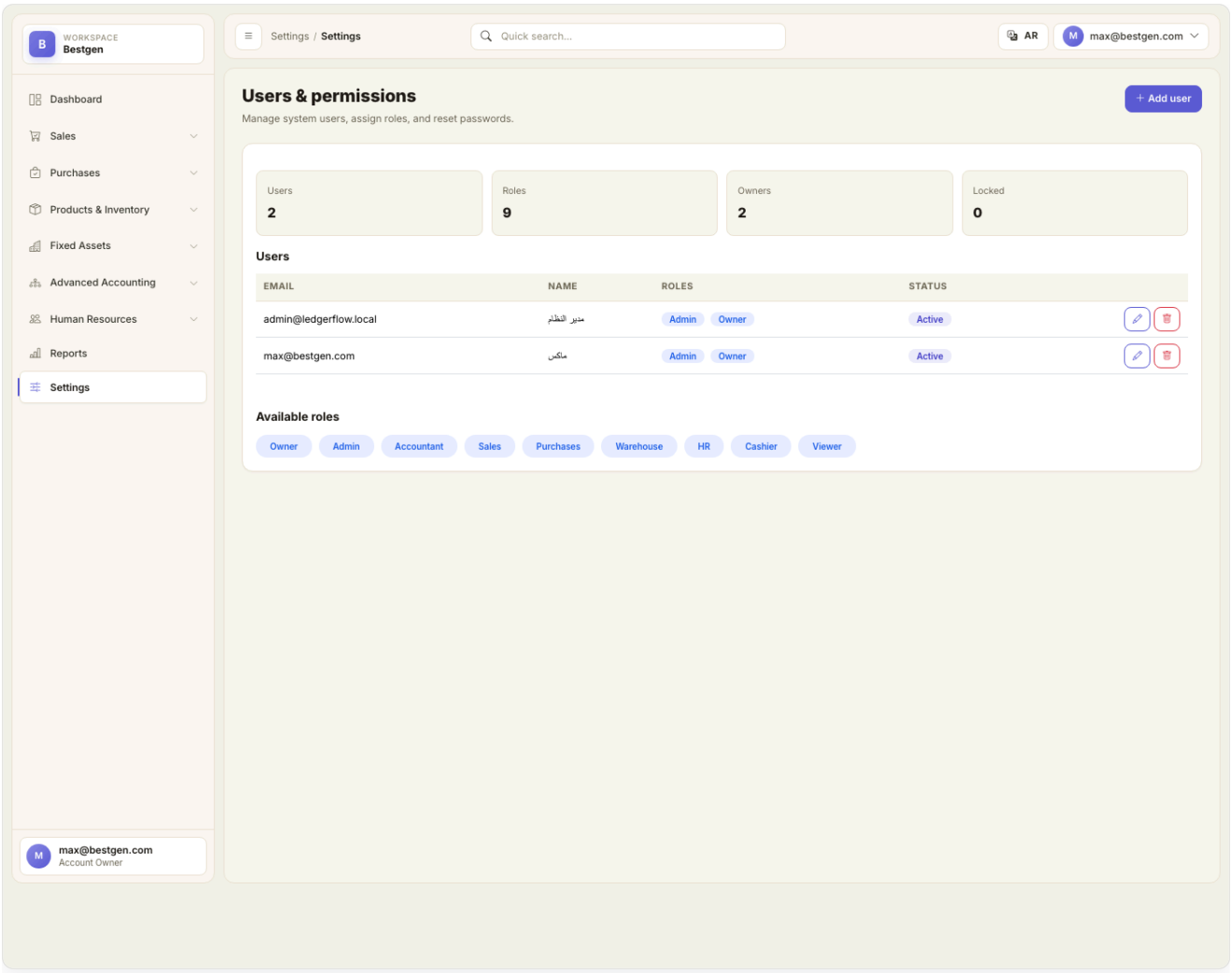


Figure 11 — Users & permissions. Role badges, lock state, create / edit / delete actions wired through ASP.NET Core Identity.

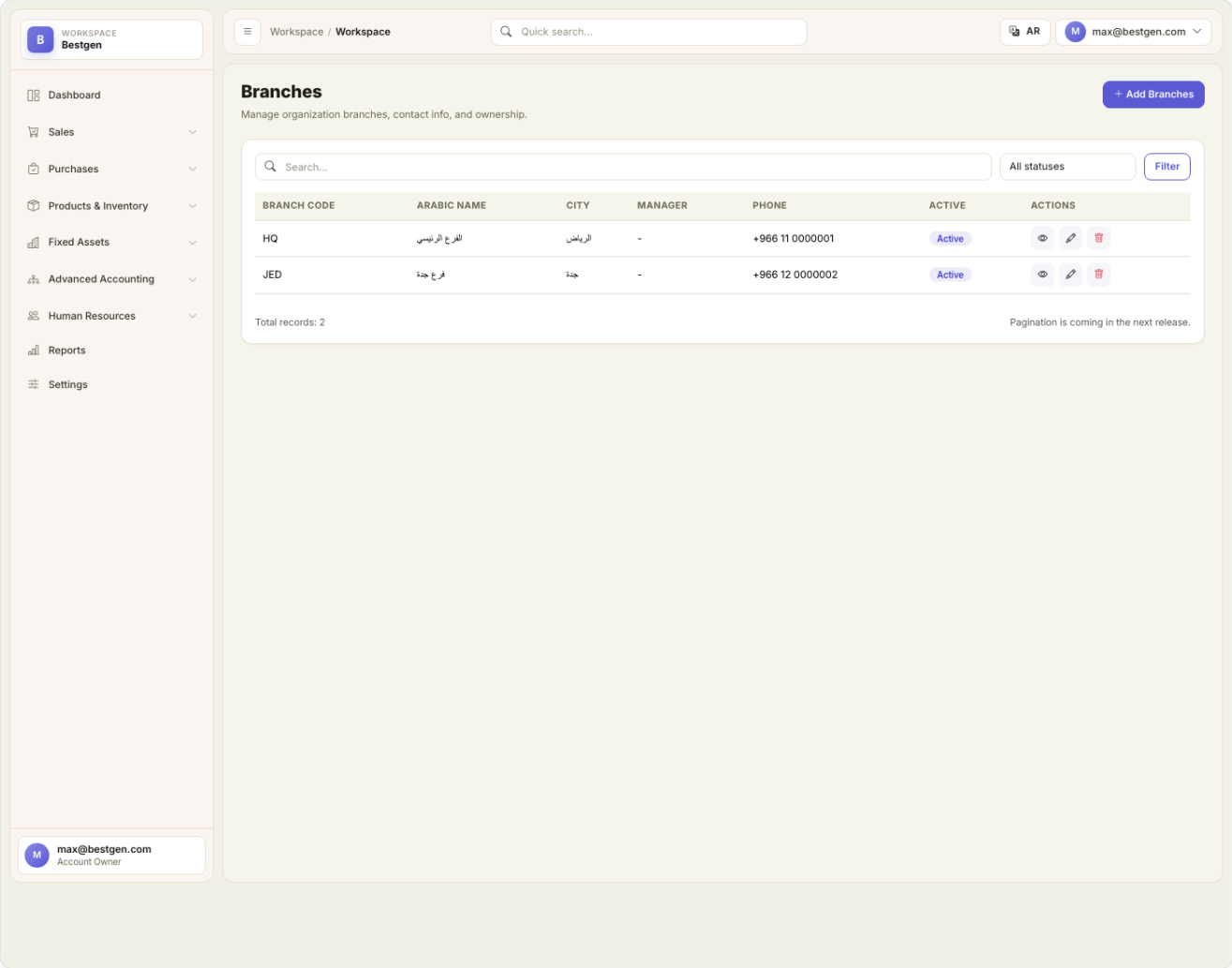


Figure 12 — Branches. Auto-rendered through `CrudController<Branch>` — no controller code needed beyond the 5-line subclass.

9 · Data model & chart of accounts

Entity inventory (high-level)

Master data

Customer · Supplier · Product · ProductCategory · Warehouse · Employee · AssetTag · Branch.

Sales documents

SalesQuotation(+Items) · SalesInvoice(+Items) · SalesReceipt · SalesRefundReceipt · CreditNote · DeliveryNote(+Items) · SalesPricePolicy.

Purchase documents

PurchaseOrder(+Items) · PurchaseInvoice(+Items) · SupplierPayment · PurchaseRefundReceipt · DebitNote · GoodsReceipt(+Items) · PurchasePricePolicy.

Inventory ops

InventoryCount(+Items) · StockTransfer(+Items) · StockMovement (audit trail).

Accounting

Account · JournalEntry(+Lines) · GeneralReceipt · OpeningBalance · ReportingDimension.

HR

Employee · EmployeeDocument · PayrollEntry · EmployeeDeduction · EmployeeBonus · EmployeeLoan · EmployeeReceipt · EmployeeRequest.

Fixed assets

FixedAsset · AssetRental · AssetTag.

Cash & banks

CashBox · BankAccount · Expense.

Configuration

CompanySettings · Branch · NumberingPolicy.

Seeded chart of accounts

Code	Account	Type	Used by
1000	Cash	Asset	All cash settlements
1010	Bank	Asset	All bank settlements
1100	Accounts Receivable	Asset	Sales invoices, receipts, credit notes
1110	AR – Employees	Asset	Employee loans
1200	Inventory	Asset	Stock-bearing flows (purchase, sale, transfer, count)
1500	Fixed Assets	Asset	Asset acquisition
1510	Accumulated Depreciation	Contra-asset	Reserved — depreciation runner pending
2000	Accounts Payable	Liability	Purchase invoices, payments, debit notes
2100	VAT Output	Liability	Sales VAT
2110	VAT Input	Asset	Purchase VAT
2200	Salaries Payable	Liability	Approved-but-unpaid payroll
2220	Loans Payable	Liability	Reserved
2230	Goods Received Not Invoiced	Liability	Reserved for goods-receipt clearing
2300	Customer Advances	Liability	Reserved
3000	Equity	Equity	—
3100	Retained Earnings	Equity	Balance sheet rollover
3200	Opening Balance Equity	Equity	OpeningBalance bootstrap
4000	Sales Revenue	Revenue	Sales invoices
4100	Service Revenue	Revenue	Reserved
4200	Asset Rental Revenue	Revenue	AssetRental
4900	Sales Returns	Contra-revenue	Refunds, credit notes
5000	COGS	Expense	Sales invoices
5100	Rent Expense	Expense	Recurring expenses
5200	Salary Expense	Expense	Payroll
5210	Bonus Expense	Expense	Bonuses
5300	Utilities Expense	Expense	Recurring expenses
5400	Misc Expense	Expense	Misc paid expenses
5700	Depreciation Expense	Expense	Reserved
5800	Inventory Variance	Expense	Inventory count adjustments
5900	Purchase Returns	Contra-expense	Refunds, debit notes

10 · Current status & what is left

Bestgen has shipped through Phase 8 of the original 10-phase production plan. Phases 0 through 8 are complete and verified by smoke tests. Phases 9 and 10 are queued.

Phase 0 · Foundation — Done

`StockMovement` audit trail · `ChartOfAccounts` resolver · `PartyBalanceService` · `BuildAndAddEntryAsync` shared posting helper · 28 chart-of-account codes seeded.

Phase 1 · Sales-side ledger wire-up — Done

SalesReceipts · SalesRefundReceipts · CreditNotes · DeliveryNotes — all post balanced journals; party balances recalculate on save.

Phase 2 · Purchase-side ledger wire-up — Done

SupplierPayments · PurchaseRefundReceipts · DebitNotes · GoodsReceipts — symmetric to the sales side.

Phase 3 · Inventory operations — Done

StockTransfers (paired movements) · InventoryCounts (variance posted) · OpeningBalances (bootstraps the books).

Phase 4 · Accounting documents — Done

`JournalEntryService` extracted from the controller · `GeneralReceiptService` with cash-account rebalancing · period-lock hook in place.

Phase 5 · HR to the ledger — Done

Payroll · Bonus · Deduction · Loan · EmployeeReceipt — all five flows post to the ledger with the right cash / payable split based on status.

Phase 6 · Fixed assets & rentals — Done

`FixedAssetService.PostAcquisitionAsync` books the asset; `AssetRentalService` generates rental numbers. Recurring rental revenue and monthly depreciation are queued for Phase 9.

Phase 7 · Reports Center — Done

Dispatcher pattern · 13 live reports · single Razor renderer · date filter wiring · placeholder fallback for the long-tail reports.

Phase 8 · Settings completion — Done

Real Tax / Invoice / Currency / Numbering / Branches forms · Identity Users CRUD · `DocumentNumberingService` · 8 default `NumberingPolicy` rows seeded · 2 default branches seeded · logo upload.

Phase 9 · Hardening Queued

- Tighten Identity password rules in `Program.cs` (post-seed `ResetPasswordAsync` keeps dev login alive).
- `[Authorize(Roles = "...")]` on every controller per the role matrix.
- `[Timestamp] byte[] RowVersion` on financial entities for optimistic concurrency.
- `ServiceResult<T>` with validation error list returned to controllers.
- `CreatedBy` / `UpdatedBy` audit log via `SaveChangesInterceptor`.
- Period locking — block journals dated before `CompanySettings.LockedThrough`.
- `MonthEndService` hosted job for depreciation runs and aging snapshots.

Phase 10 · Tests & release Queued

- xUnit project at `tests/Bestgen.Tests/` with EF Core in-memory provider.
- End-to-end coverage: sales invoice posts balanced journal + reduces stock, purchase invoice + payment, stock transfer pairs to zero, period lock blocks back-dated entries, `IsBalanced` rejects unbalanced journals, every report against a seeded fixture.
- `tests/SMOKE.md` 30-minute click-through manual checklist before each release.

Out-of-scope (deliberate)

- **EF Migrations** — currently using `EnsureCreatedAsync`; should be replaced with proper migrations before going to production with real customer data.
- **Multi-currency & FX** — base currency is single-tenant; multi-currency wallet not modeled.
- **ZATCA Phase 2 clearance** — taxpayer ID and QR toggle exist; the actual FATOORA submission integration is on the roadmap.
- **EF Migrations / multi-tenant tenancy** — Bestgen is single-tenant by design at this stage.

11 · Operating the application

Day-zero bootstrap

1. Clone the repository and run `dotnet restore`.
2. Run `dotnet run` — the seeder creates the SQLite file, seeds 9 roles, 28 accounts, 2 admin users, 2 branches, 8 numbering policies, plus demo data.
3. Visit `http://localhost:5000` and sign in with `max@bestgen.com / 123`.
4. Open *Settings* → *Company profile* and replace the seeded company name, VAT number, commercial registration, and logo.
5. Open *Settings* → *Users & permissions* and add real users with the appropriate roles.

Day-to-day flows

- **Sell something** — Customers → Sales Invoices → New. Lines, VAT, totals, journal, and stock all process in one save.
- **Get paid** — Sales Receipts → New. Pick the customer and the cash/bank account.
- **Buy something** — Suppliers → Purchase Invoices → New. Same multi-line shape as the sales side.
- **Pay a supplier** — Supplier Payments → New.
- **Reconcile a count** — Inventory Counts → New, mark *Approved* — variance is posted automatically.
- **Look at the books** — Reports → Trial Balance / Income Statement / Balance Sheet — all live.

Typical pitfalls & how Bestgen catches them

Pitfall	Built-in guard
Unbalanced journal	`AccountingService.BuildAndAddEntryAsync` throws if debits ≠ credits.

Pitfall	Built-in guard
Missing chart-of-account code	`ChartOfAccounts.ResolveAsync` throws `MissingAccountException`.
Stock mutation outside the audit trail	Nothing else writes to `Product.CurrentStock` — every change goes through `StockMovementService`.
Stale party balance	`PartyBalanceService.RecalculateCustomer/SupplierAsync` recomputes from primary docs after every relevant save.
Duplicate document number	Unique indexes on every numbered document type — saved at `OnModelCreating`.

12 · Conclusion

In one sentence: **Bestgen** is a complete, bilingual accounting and inventory product engineered for Saudi small and mid-sized businesses. It solves three problems at once that owners face daily — scattered books across Excel sheets, no clear visibility into actual warehouse stock, and the time lost preparing tax and financial reports — by replacing them with one cohesive system that ships with proper double-entry accounting from day one.

What Bestgen does

Runs the full business cycle: issues quotations, posts invoices, records payments, deducts stock, and writes the journal entry to the general ledger — all in a single save.

Why it's different

The UI is Arabic-native, not translated. The ledger is properly double-entry. Reports query the live database — no mocked figures. Identity and roles are built in from day one.

Who it's for

The bookkeeper, the warehouse manager, the sales clerk, the purchasing officer, and the HR clerk in a small or mid-sized company — each finds their section ready in the sidebar.

Where it stands today

9 sidebar sections complete. 9 roles. 28 chart-of-account codes. 13 live reports. ~70 entities in the database. 8 of 10 production-plan phases shipped.

What's left to ship

Three clusters of work: tighten password rules and per-controller role gates, add the period-lock guard, and write the automated test suite. Then replace `EnsureCreatedAsync` with proper EF migrations before the first real customer.

When it's deployable

Today, for in-house or private-server use. After Phases 9 and 10, it's ready for commercial release on real customer tenants.

> **The takeaway in the simplest terms:** Bestgen has crossed the line from scaffold to working product. Every save you make moves the ledger, the stock, and the party balance together. You can already pull real accounting and inventory reports out of it. What remains is a final hardening pass before commercial deployment — not a rebuild.